

무엇을 만드는 도구인가요?

원본 구조를 유지하면서 색감과 장식으로 분위기를 바꾸는 게임 에디터

01 현재 대상과 구현 범위

Crypt 1층 폐허마을(floor-1-ruins)의 소품 스타일 편집 도구입니다. 크리스마스/겨울의 눈·전구와 고정 겨울 팔레트 보정을 지원합니다. 현재 DSL-Planner는 자체 규칙 기반이며, 자유로운 모든 테마를 이해하는 AI가 아닙니다.

02 제공하는 기능

프롬프트 해석과 대상 선택, FLUX 장식 생성, 원본/결과/장식 비교, 변경 목록, 명시적 게임 적용, 적용 상태 저장, 원본 보기 토글, 실험 기록을 제공합니다. 게임 에디터와 스타일 페이지는 차콜·블루 UI로 통일했으며 기존 사이드바 호버 미리보기를 유지했습니다.

03 가장 중요한 원칙

원본 TMX, 형태, 크기, 문·창문 위치, 베이스 투명도와 충돌 좌표를 바꾸지 않습니다. 생성된 그림 전체를 원본 대신 붙이지 않고, 눈·전구 등 필요한 장식만 추출해 별도 계층에 합성합니다. 색·명암 변경을 요청한 경우에만 베이스 색을 보정합니다.

04 현재 결과와 진행 중 작업

2중·126곳의 개선 장식 합성은 게임 적용과 새로그침·토글 검증을 마쳤습니다. 전체 45종·1,193곳의 신규 배치는 문서 작성 시 13/45종 생성 중이며 미적용입니다. 이 수치는 실시간 진행률이 아닙니다. 기존 2026-09-10 겨울 맵 결과와 이번 신규 배치를 구분해야 합니다.

변환은 어떻게 동작하나요?

프롬프트 → 계획 → 생성 → 추출·합성 → 검수·적용

01 TMX 기반 원본 구성

기존 원본 크롭과 TMX 배치 좌표·소스 해시를 읽고 연결 요소 기반 소품 그룹을 구성합니다. 동일한 소품 이미지는 한 번 생성한 뒤 여러 위치에 재사용합니다. 현재 prop 그룹에 집·나무 같은 의미 레이블을 자동으로 붙이지 않으며, 이 경로는 SAM 기반 분류가 아닙니다.

02 DSL-Planner와 FLUX

dsl.py는 눈·전구·색 보정 요구를 검증하고 planner.py는 그룹 크기와 구조 보존 조건을 포함한 개별 프롬프트를 작성합니다. FLUX.1-Kontext-dev 서비스는 28 steps, guidance 2.5, BF16, CPU offload 설정입니다. 프로젝트의 핵심은 추가 학습이 아닌 프롬프트와 후처리이며, 서버의 추가 가중치 로딩 여부는 별도 확인이 필요합니다.

03 장식 알파와 좌표 보정

생성 결과에서 눈·전구 색상과 원본 주변 범위를 이용해 장식을 추출합니다. 베이스 알파는 그대로 두고 장식에는 최대 3px 여백을 허용합니다. 16×16 원본의 경우 22×22 캔버스에 (-3,-3) 오프셋을 적용해 원래 좌표를 유지합니다. 마스크는 휴리스틱이므로 잔여 배경과 미적 품질은 검수해야 합니다.

04 누적 오류 방지와 적용

composition_base_id와 asset_revisions로 기존 결과부터 재합성해 이전 장식 여백이 계속 쌓이지 않게 합니다. 계약·소스·알파와 부모 결과를 검증하고, 사용자가 적용을 눌렀을 때만 선택 상태를 변경합니다. 다른 층과 원본 파일은 유지합니다.

사용 방법

생성 서버는 유지하고, 화면에서 결과를 검토한 다음 적용하세요

01 새 환경에서 준비

Node.js 20.19+ 또는 22.12+, Python, Git LFS가 필요합니다. 저장소에서 `git lfs pull`, `npm ci`를 실행하고 Python의 `fastapi`, `uvicorn`, `pillow`, `numpy`, `requests`를 설치합니다. `python scripts/restore-crypt-ruins.py --apply`로 보관된 1층 기준 결과를 복원합니다. 기존 내용이 다르면 덮어쓰지 않고 중단합니다.

02 서버와 화면 실행

FLUX 서버를 별도로 연결하고 `THEME_FLUX_URL`을 서비스 주소로 지정한 뒤 `npm run theme:dev`를 실행합니다. 다른 터미널에서 `npm run dev -- --host 127.0.0.1 --port 15174 --strictPort`를 실행합니다. GPU 연결이나 인증 정보는 저장소 복제만으로 제공되지 않습니다.

03 프롬프트 입력부터 적용까지

게임 에디터의 스타일 변환 → 프롬프트 입력 → 대상 확인 → 대상 체크(기본 2종 또는 전체 선택) → 스타일 생성하기 → 원본/결과/장식 검수 → 게임에 적용하기 순서입니다.

예시: 크리스마스 밤. 원본 색은 그대로 두고 눈과 전구를 추가하고 반짝이게 해줘.

장식 표시 스위치로 원본과 비교하고 이전 실험 기록에서 과거 결과를 다시 엽니다. 고급 설정에는 계약 JSON과 Visual DSL이 있습니다.

04 진행 중 주의사항

화면 새로고침은 서버 작업을 취소하지 않으며 스타일 페이지는 실행 중 배치를 먼저 표시합니다. Python 생성 서버 재시작은 피하세요. 45종은 1,193회 생성이 아닙니다. 앞선 두 소품 요청이 각 약 203/204초였으므로 순차 45종은 약 2~3시간 예상이며 크기·서버 상태에 따라 달라집니다.

검증·저장·남은 작업

기능 구현 완료와 전체 결과 품질 승인은 서로 다른 단계입니다

01 확인한 내용

Python 서비스 테스트 78개, 배치 저장 테스트 12개, TypeScript/Vite 빌드를 통과했습니다. 적용된 126곳에서 다른 추출 오브젝트·선택 장식 간 겹침과 맵 밖 잘림 검출은 0건입니다. 벽·캐릭터·상속 장식과의 가림, 시각적 완성도는 이 검사에 포함되지 않습니다. 빌드에는 CSS import 순서와 번들 관련 경고가 남아 있습니다.

02 기록과 백업 정책

public/theme-runs/실행ID에 입력·프롬프트·원본 생성 이미지·장식·미리보기·시간·이벤트를 보존합니다. 재처리하는 새 ID와 출처를 기록하며 과거 실험을 덮어쓰지 않습니다. 사용자가 실패라고 지정한 것만 별도 실패 보관 대상으로 삼습니다. 작은 실패본을 서버 한곳에 자동 수집하는 기능은 아직 미구현입니다.

03 GitHub에 포함되는 것

코드·문서와 기존 Git LFS 보관 결과를 포함합니다. 현재 실행 폴더와 적용 상태는 Git 제외이며 신규 배치 결과는 이번 업로드에 포함하지 않습니다. 생성 종료 후 `python scripts/archive-theme-runs.py --output 백업폴더`로 무손실 ZIP과 해시 검증을 수행할 수 있습니다. 실행 중에는 이 백업 명령을 거부합니다.

04 남은 단계

신규 45종 배치 완료 → 전체 결과 품질·가림 검수 → 명시적 게임 적용 → 완료 시점 백업이 남아 있습니다. 의미 기반 오브젝트 분류, 다양한 테마·시간대, 바닥·벽 재테마, 여러 후보 선택과 seed 제어는 확장 과제입니다. 현재 기능을 이 범위까지 완료했다고 해석하면 안 됩니다.

참고 위치

README.md: 최신 상태와 실행 명령

docs/prompt-theme-integration.md: 구현·계약·제약

docs/placement-audit.md: 자동 검수 범위

GitHub: <https://github.com/yc0153/kong/tree/style>